Ficha de Trabalho — Aula 4

Comunicação entre sistemas: escolher a abordagem adequada

1. Desafio

Abre a cena `Assets/Scenes/Aula4_ComunicacaoSistemas.unity` no projeto `pdj2-comunicacao-sistemas` e executa-a em Play Mode. A cena inclui um inimigo reutilizado conscientemente da aula anterior: serve para demonstrar que partes de um projeto existente podem ser reaproveitadas após confirmar que continuam adequadas; não tens de o reimplementar. Se o ativares em Play Mode, ele causa dano quando apanha o Player e apresenta Game Over quando a vida chega a zero. Esta mecânica é funcional, mas não faz parte da implementação pedida nesta ficha.

A tarefa principal é implementar a comunicação da recolha: o jogador interage com um item, a pontuação aumenta, a interface em UI Toolkit recebe a alteração e uma porta reage quando a pontuação atinge o valor necessário.

Não são necessários assets finais. Usa os GameObjects e os elementos de UI Toolkit preparados pelo professor. O objetivo é produzir e testar código, não construir uma apresentação visual.

2. Objetivos

No final do trabalho deverás conseguir:

- implementar uma capacidade através de uma interface;
- distinguir uma chamada de método de uma notificação por evento;
- explicar que a interface define o contrato usado numa chamada, não substitui a chamada;
- fazer vários sistemas reagirem sem o emissor os conhecer diretamente;
- justificar por que a referência explícita é suficiente neste exercício e identificar o custo de a substituir por acesso global via Singleton;
- testar e explicar as dependências da solução.

3. Regras de trabalho

- Usa a cena de teste preparada `Aula4_ComunicacaoSistemas` ; não alteres `SampleScene` .
- Faz os testes em Play Mode; não é necessário criar uma build.
- Segue os objetos já configurados na cena; concentra o trabalho nos scripts e nas dependências entre sistemas.
- Implementa por passos pequenos; confirma que o projeto compila antes de prosseguir.
- Não cries uma interface ou evento sem ligares a escolha a um requisito. A implementação obrigatória usa referência explícita; Singleton é apenas comparação/reflexão.
- Não coloques a lógica de pontuação na interface nem a lógica da porta no item.
- Usa a Console apenas para diagnóstico técnico. A pontuação e o ecrã de vitória devem usar UI Toolkit; o inimigo reutilizado pode conservar a UI legada da aula anterior, que não faz parte desta implementação.
- Os collectibles usam colliders trigger para poderem ser tocados pelo Capsule Collider do Player sem bloquearem o movimento. O trigger da área de interação também pode detetar o item; impede que entradas sobrepostas contem como recolhas repetidas.
- Durante a aula, prepara o teu fork, clone local e ramo de trabalho. Depois, durante a semana e até à data indicada pelo docente, envia as alterações e abre um Pull Request seguindo o guia de entrega.
  O fork e o Pull Request são públicos; não incluas palavras-passe, tokens ou dados pessoais.

Checkpoints de trabalho autónomo

1. Faz o Player encontrar um `IInteractable` e chama `Interact` . Confirma que compila antes de passar à pontuação.
2. Faz o item pedir pontos ao `ScoreSystem` atribuído no Inspector e impede recolhas repetidas.
3. Atualiza a pontuação antes de emitir `ScoreChanged` ; implementa o subscritor do HUD e completa `ScoreDoor` para reagir ao limiar.
4. Testa a leitura do valor inicial, a remoção das subscrições e os casos da tarefa D. Depois, se concluíres o essencial, faz o exercício extra do ecrã de vitória.

Em cada checkpoint, prevê o resultado, executa o teste e regista o que observaste. Se ficares bloqueado, identifica primeiro o objeto, a referência ou o erro que suspeitas; depois pede uma pista ao professor.

4. Modelo teórico de referência

Antes de implementar, consulta a apresentação da aula. Usa-a para justificar as escolhas; Singleton e Event Queue não são requisitos da solução principal.

Usa estas distinções para analisar a tarefa. Não são quatro soluções concorrentes:

Conceito	Pergunta que ajuda a responder	Papel
Chamada direta	Quem deve executar esta operação agora?	Pedido síncrono a um destinatário conhecido
Interface	Que capacidade/contrato precisa o consumidor?	Tipo através do qual faz a chamada, sem depender da classe concreta
Evento	Quem deve ser informado de que um facto já ocorreu?	Notificação a zero ou vários subscritores
Singleton	Como é obtida uma instância que se pretende única/global?	Estratégia de acesso, não mecanismo de comunicação

Um evento C# é normalmente síncrono: os handlers executam durante a emissão. Não equivale a uma fila de mensagens nem guarda um histórico para subscritores tardios. Uma subscrição cria uma dependência de ciclo de vida: deve ser removida quando o componente deixa de estar ativo.

5. Tarefa A — Interação através de um contrato

Cria ou completa `IInteractable` com uma operação de interação. Implementa o contrato num `Collectible`.

O consumidor da interação deve:

- detetar o objeto com que o jogador pode interagir;
- trabalhar com o contrato `IInteractable`, e não depender da classe concreta `Collectible`;
- chamar a interação uma única vez para cada recolha válida.

Testa: recolhe os dois itens preparados na cena. Ambos devem funcionar através do mesmo contrato. Como extensão, cria um segundo tipo de objeto interativo e confirma que o consumidor continua a funcionar sem passar a depender da nova classe concreta.

6. Tarefa B — Atualizar a pontuação

Implementa um sistema responsável pela pontuação. Quando o item é recolhido:

- o item deixa de poder ser recolhido novamente;
- a pontuação aumenta pelo valor configurado;
- o sistema publica a nova pontuação depois de atualizar o estado.

Usa a referência explícita ao `ScoreSystem` atribuída no Inspector. Regista por que esta dependência é adequada ao exercício. Não implementes um Singleton como parte obrigatória da ficha; a comparação com acesso global é conceptual ou uma extensão opcional.

7. Tarefa C — Reagir à alteração

Implementa os dois subscritores:

- um componente que atualiza o `Label` `score-label` no UI Toolkit;
- `ScoreDoor`, que reage à pontuação e abre quando atinge o limiar definido.

Requisitos:

- os subscritores não são chamados diretamente pelo item;
- o evento é emitido depois de a pontuação mudar;
- ambos removem os handlers quando deixam de estar ativos;
- a apresentação lê o estado atual ao iniciar, mesmo que tenha subscrito depois da primeira alteração.

Ao abrir, a porta deve continuar ativa, mas deixar de bloquear a passagem: marca o collider como trigger e desativa o `MeshRenderer`. Não desatives o GameObject, porque ele tem de receber `OnTriggerEnter`.

8. Tarefa D — Testes

Executa e regista estes testes:

Teste	Resultado esperado	Resultado observado
Recolher cada item preparado	Cada item é desativado e a pontuação aumenta uma vez	

Teste	Resultado esperado	Resultado observado
Ter `ScoreDisplay` e porta subscritos	Ambos reagem à mesma alteração	
Atingir pontuação inferior ao limiar	A porta mantém-se fechada	
Atingir o limiar	A porta fica invisível, o collider passa a trigger e o GameObject mantém-se ativo	
Desativar apenas o componente `ScoreDisplay` e alterar a pontuação	O Label deixa de atualizar e a porta continua a reagir	

9. Exercício extra — ecrã de vitória ao atravessar a porta

Depois de a pontuação abrir a porta, o Player deve conseguir atravessá-la. A porta muda o seu collider para trigger e desativa o `MeshRenderer`. Ao entrar no trigger, a porta emite `OnDoorEntered`. Completa essa parte da porta e cria e configura a reação visual.

1. No UXML usado pelo `UIDocument`, cria um painel chamado `victory-panel`, com um texto de vitória. Associa uma classe USS e deixa o painel oculto no início.
2. Completa o `OnTriggerEnter` de `ScoreDoor` para filtrar o Player e emitir `OnDoorEntered` sempre que este entra no trigger. O Player raiz tem de ter a tag `Player`.
3. Cria um componente de apresentação, por exemplo `VictoryScreen`, que obtenha o painel pelo nome, subscreva a `ScoreDoor.OnDoorEntered` e o torne visível quando recebe a notificação.
4. Atribui a porta no Inspector e remove a subscrição quando o componente deixa de estar ativo.

Testa: antes do limiar, a porta bloqueia a passagem e não aparece o ecrã. Depois do limiar, atravessa a porta e confirma que o painel aparece. Sai do trigger e volta a entrar: a porta deve notificar novamente. Para observar cada entrada, usa temporariamente um registo de diagnóstico no subscritor ou esconde o painel entre tentativas.

O painel é responsabilidade de apresentação; a porta não deve procurar o `UIDocument` nem controlar diretamente a interface. O evento comunica o facto de o Player ter entrado. Remove os handlers no ciclo de vida adequado.

10. Conclusão individual

Escreve 4–6 frases sobre o que acontece no teu projeto. Responde a estas perguntas:

- Quando o Player toca num item, que interface permite ao sistema de interação chamar o item?
- Quem aumenta a pontuação? Depois dessa alteração, que sistemas recebem o evento e como reagem?
- Desativa o componente `ScoreDisplay` e recolhe os dois itens. O que acontece à pontuação, ao HUD e à porta? O que te mostra este teste sobre a forma como estes sistemas comunicam?
- Nesta cena, cada item tem uma referência ao `ScoreSystem` no Inspector. Porque é que isso é suficiente? Imagina que havia duas salas, cada uma com a sua própria pontuação: que problema poderia surgir se ambas usassem o mesmo Singleton global?

11. Leituras e bibliografia

As fontes abaixo aprofundam conceitos gerais; não é necessário ler todas antes da ficha.

Para começar

- Microsoft Learn, Interfaces em C#.
- Microsoft Learn, Eventos em C#.
- Robert Nystrom, *Game Programming Patterns*: Observer, Decoupling Patterns e Singleton. O livro está disponível gratuitamente para leitura online; os exemplos são maioritariamente em C++, mas os conceitos transferem-se para C#.

Para consultar durante a implementação

- Unity, Execution order of event functions, para o ciclo de vida e ordem dos callbacks de `MonoBehaviour`.
- Microsoft Learn, Handling and raising events, para o modelo de publicação/subscrição em .NET.

Aprofundamento opcional

- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. Referência clássica do padrão Observer.
- Robert C. Martin, The Single Responsibility Principle e Design Principles, para SRP e DIP.

Ao consultar uma fonte, identifica que pergunta te ajuda a responder — por exemplo, "quando preciso de desacoplamento temporal?" — em vez de tentares memorizar definições isoladas.

12. Critérios de sucesso

- [ ] O projeto compila sem erros.
- [ ] O consumidor interage com os itens através de `IInteractable`, sem depender de `Collectible`.
- [ ] A pontuação é gerida fora do item e da apresentação.
- [ ] O evento é emitido depois de a pontuação ser atualizada.
- [ ] O Label de pontuação em UI Toolkit e a porta reagem independentemente.
- [ ] As subscrições são removidas no ciclo de vida apropriado.
- [ ] Os testes da ficha foram executados.
- [ ] A conclusão explica, com exemplos da cena, como os sistemas comunicam e justifica a referência escolhida para o `ScoreSystem`.
- [ ] **Exercício extra:** o painel de vitória aparece ao entrar na porta e a UI subscreve ao evento sem ser controlada diretamente pela porta.